Parallel Task Scheduler

Comparative Performance Analysis of 5 Scheduling Algorithms
Implemented in C++ with OpenMP Parallelism

Made by :
Bipasha (102303388)
Suneet Singla (102303615)

FCFS · Round Robin · MinMin · MaxMin · LoadAware

Submitted to:
Dr. Saif Nalband

Project Overview

Objective · Technology · Experiment Design

Objective

Design and evaluate a parallel task scheduler using DAG-based workloads, comparing 5 scheduling algorithms across multiple configurations to identify the most efficient strategy.

Key Idea

Tasks are interdependent (DAG) and must be scheduled across multiple processors efficiently.

Build & Run

`make run` → `./scheduler` → `results.csv` → `plot.py`

Experiment Setup

- Task Sizes: 50, 100, 200
- Processors: 2, 4, 8
- Total Configurations: 9

Approach

- Generate tasks with dependencies
- Apply different scheduling algorithms
- Execute in parallel using OpenMP
- Measure performance using 4 key metrics

Tech Stack

C++ (OpenMP), Python (matplotlib), CSV pipeline

System Architecture and Key Concepts

Task Generator

Generates tasks with random execution times and dependencies

DAG Constructor

Builds dependency graph ensuring no cycles

Scheduler Module

Applies algorithm (FCFS, RR, Min-Min, Max-Min, Load-Aware)

Parallel Execution Engine

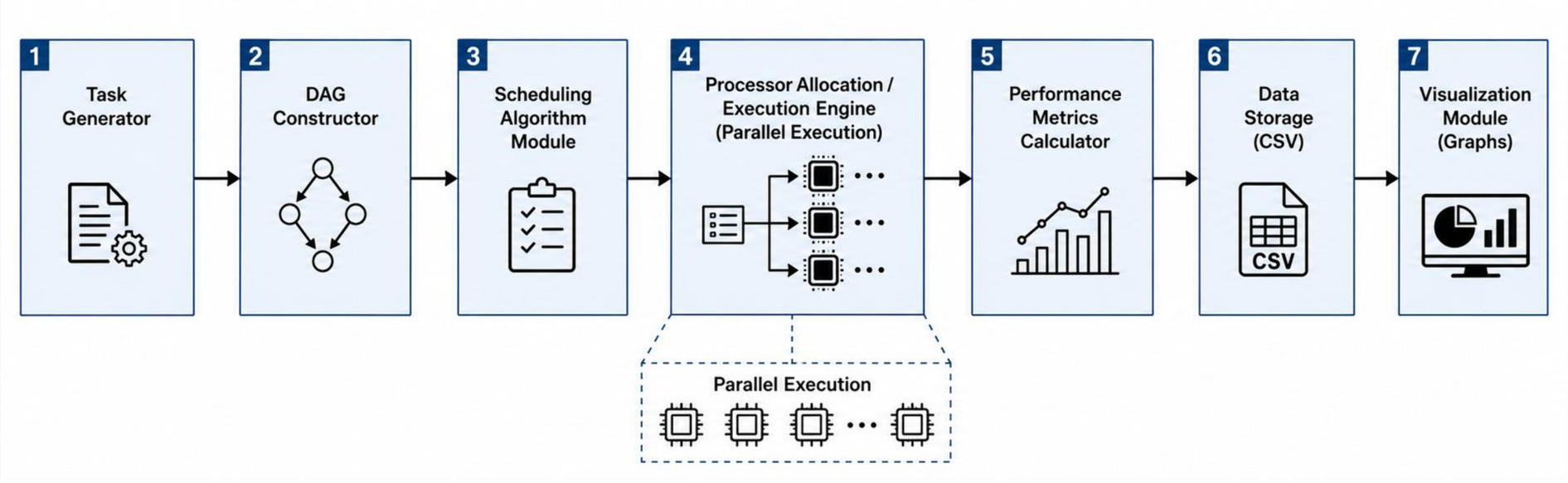
Executes tasks using OpenMP across processors

Metrics Calculator

Computes makespan, utilization, speedup, imbalance

Data Storage + Visualization

Stores results in CSV and generates graphs



Performance Metrics — Definitions & Formulas

Makespan

↓ Lower is better

```
max( finish_time[i] )  for all tasks i
```

Total elapsed time from start until the last task completes. The primary throughput metric — lower is better.

Speedup

↑ Higher is better

```
Sequential_Time  /  Makespan
```

How many times faster parallel execution is vs. running all tasks sequentially on one processor.

CPU Utilization

↑ Higher is better

```
[ Σ (busy_time[p] / Makespan) ] / P  × 100%
```

Average fraction of time each processor does useful work. Low utilization means processors are sitting idle.

Load Imbalance

↓ Lower is better

```
( max_busy − min_busy )  /  avg_busy
```

How unevenly work is distributed. 0 = perfect balance. High values mean some processors are overloaded while others idle.

Scheduling Algorithms

How each algorithm assigns tasks to processors

FCFS

How: Tasks assigned in original ID order. Processor = task_index % P. No sorting.

✅ Zero overhead, predictable

⚠️ Ignores task length & load

Round Robin

How: A counter cycles 0 → P-1 → 0. Each task gets the next processor in rotation.

✅ Even distribution by count

⚠️ Ignores execution time differences

MinMin

How: Ready tasks sorted ascending by execution_time. Shortest task scheduled first.

✅ Reduces idle wait times

⚠️ Can delay long critical tasks

MaxMin

How: Ready tasks sorted descending by execution_time. Longest tasks start first.

✅ Overlaps long + short tasks

⚠️ High load imbalance in practice

LoadAware

How: MinMin sorting, but always assigns to getLeastBusyProcessor() — the one with smallest free_time.

✅ Best balance, adapts dynamically

⚠️ Critical section serializes lookup

Load-Aware Algorithm: Core Logic

Instead of assigning tasks blindly, LoadAware selects the processor with the least current workload.

→ Dynamic decision based on real-time processor state
→ Minimizes idle time and imbalance

Algorithm Steps

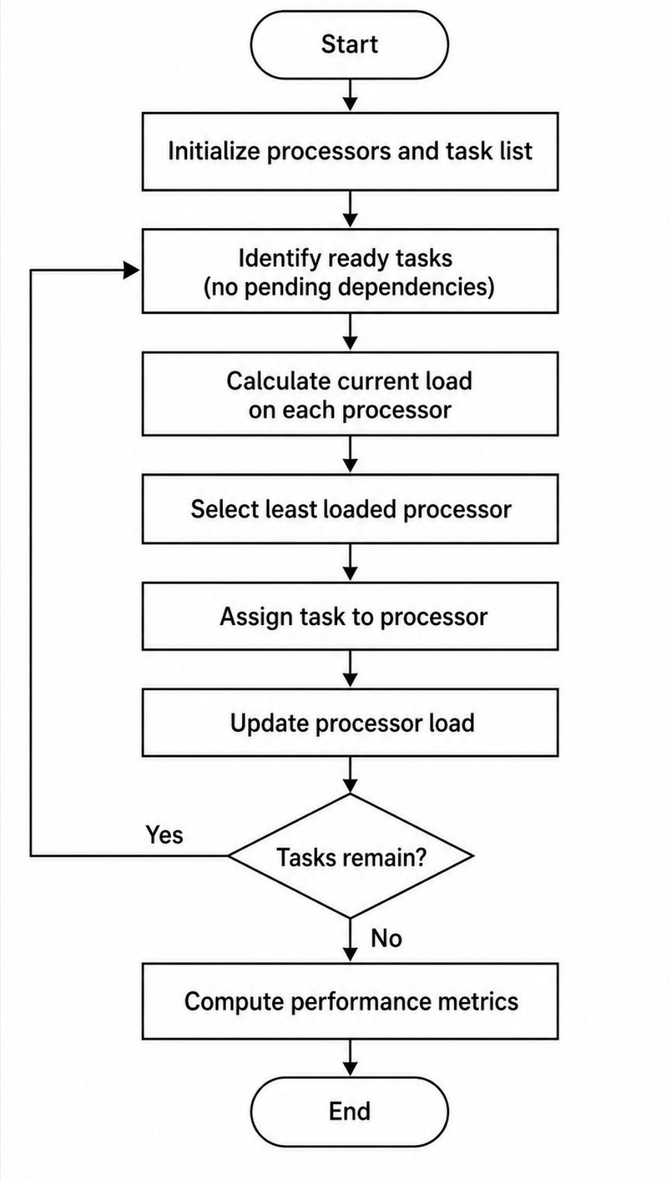
1. Identify Ready Tasks: Select tasks with no pending dependencies
2. Calculate Current Load: Sum execution times of assigned tasks for each processor
3. Find Minimum Load: Identify processor with least total workload
4. Assign Task: Allocate task to selected processor
5. Update Load: Add task execution time to processor's load
6. Repeat: Continue until all tasks are scheduled

Why it works:

- Minimizes idle time
- Balances workload dynamically
- Improves overall efficiency.

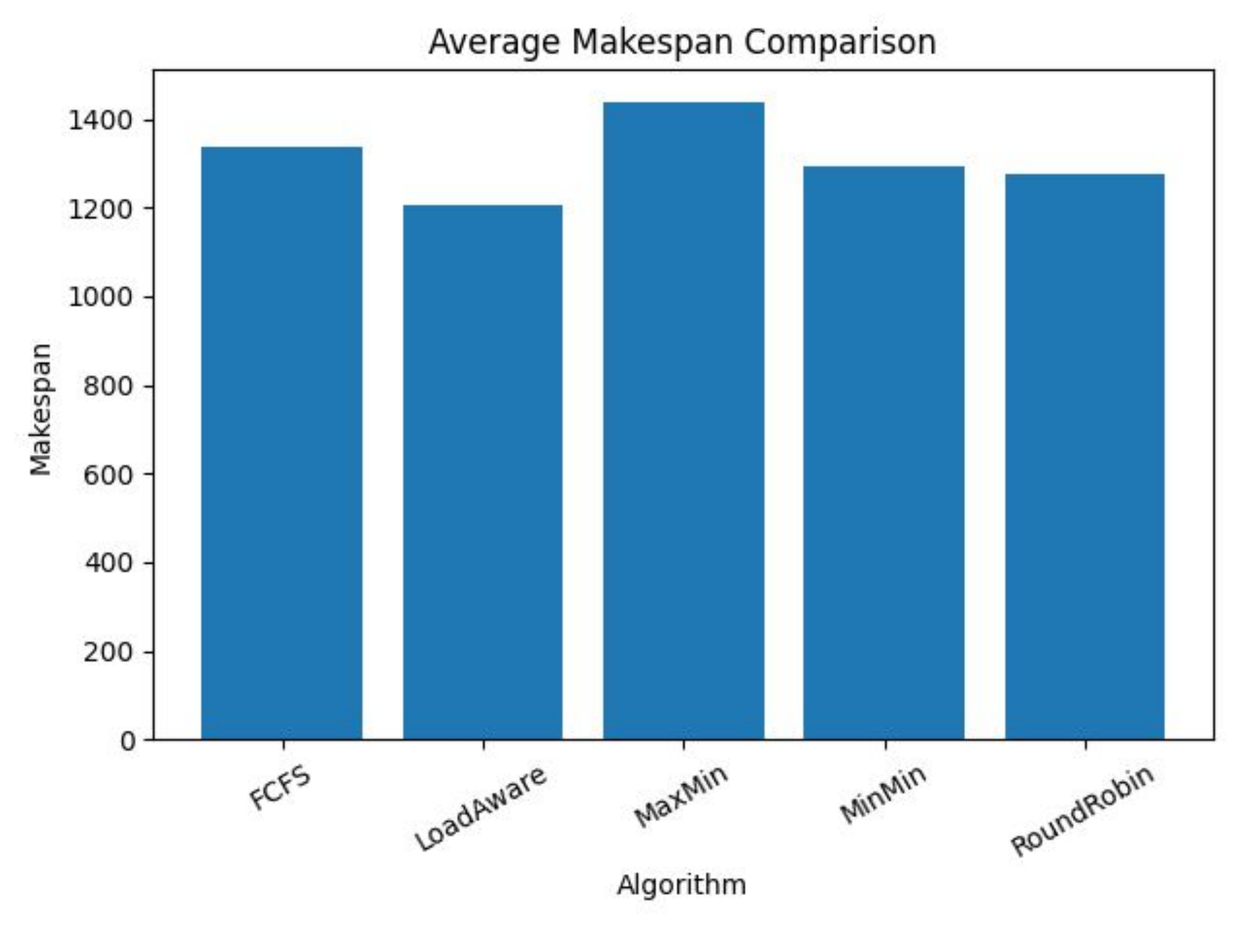
Limitations:

- Greedy approach
- Does not consider future tasksschedules.



Results — Makespan

Average time units to complete all tasks · lower is better



Best
LoadAware
1,221 units

2nd Best
RoundRobin
1,258 units

Worst
MaxMin
1,426 units

Observations

- **LoadAware** achieves the lowest average makespan, indicating efficient workload distribution across processors
- **RoundRobin** performs closely, showing stable and fair task allocation
- **MaxMin** results in the highest makespan, due to prioritizing long tasks and creating execution bottlenecks

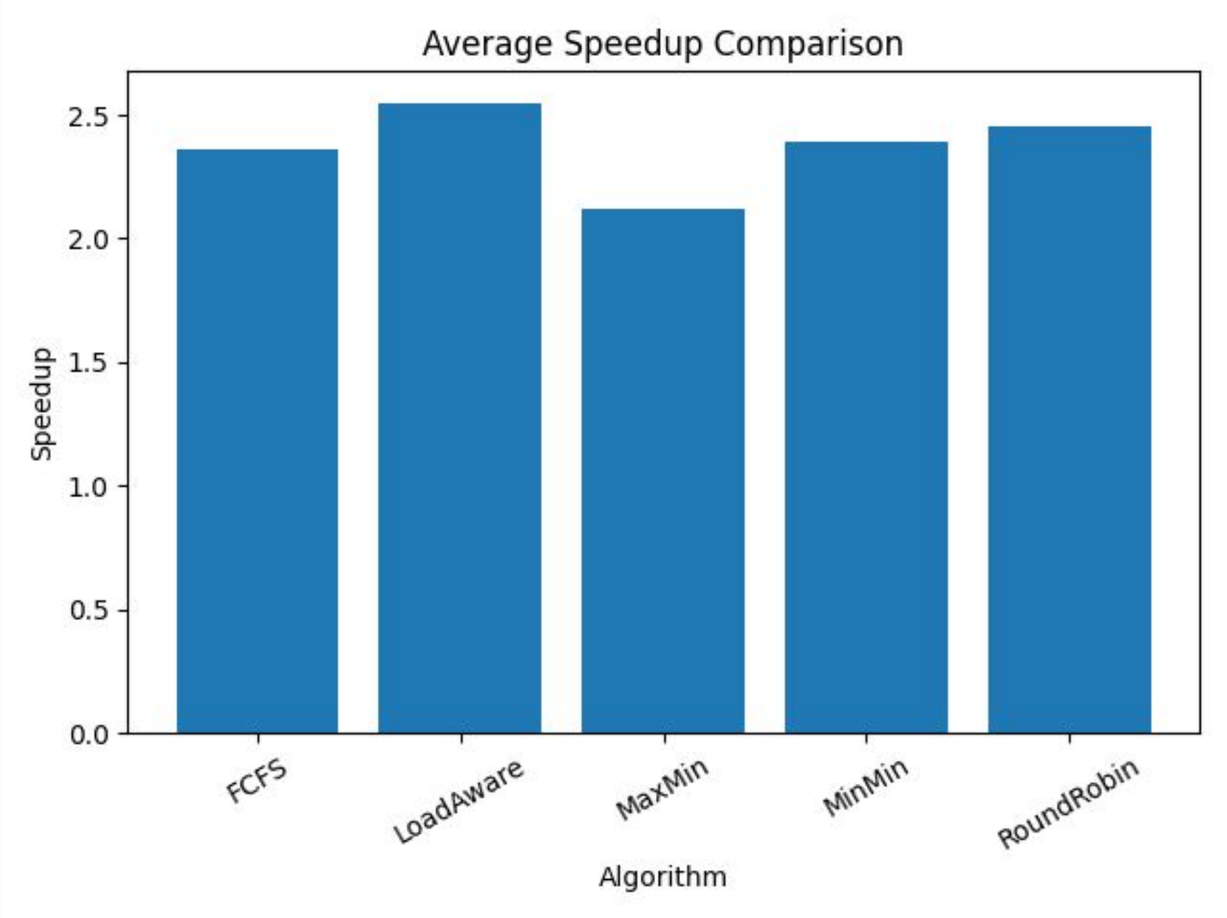
Why this happens?

- **LoadAware** minimizes idle time by balancing processor loads dynamically
- **MaxMin** delays smaller tasks, causing underutilization of processors
- **RoundRobin** ensures fairness but does not optimize based on execution time

LoadAware consistently achieves the lowest makespan by balancing workload, while MaxMin underperforms due to task prioritization imbalance. RoundRobin provides stable performance but lacks optimization.

Results — Speedup

Parallel speedup vs sequential single-processor execution · higher is better



Observations

- **LoadAware** achieves the highest speedup (~2.48×), indicating efficient parallel execution
- **RoundRobin** performs closely (~2.4×), showing stable task distribution
- **MaxMin** has the lowest speedup (~2.08×) due to inefficient scheduling

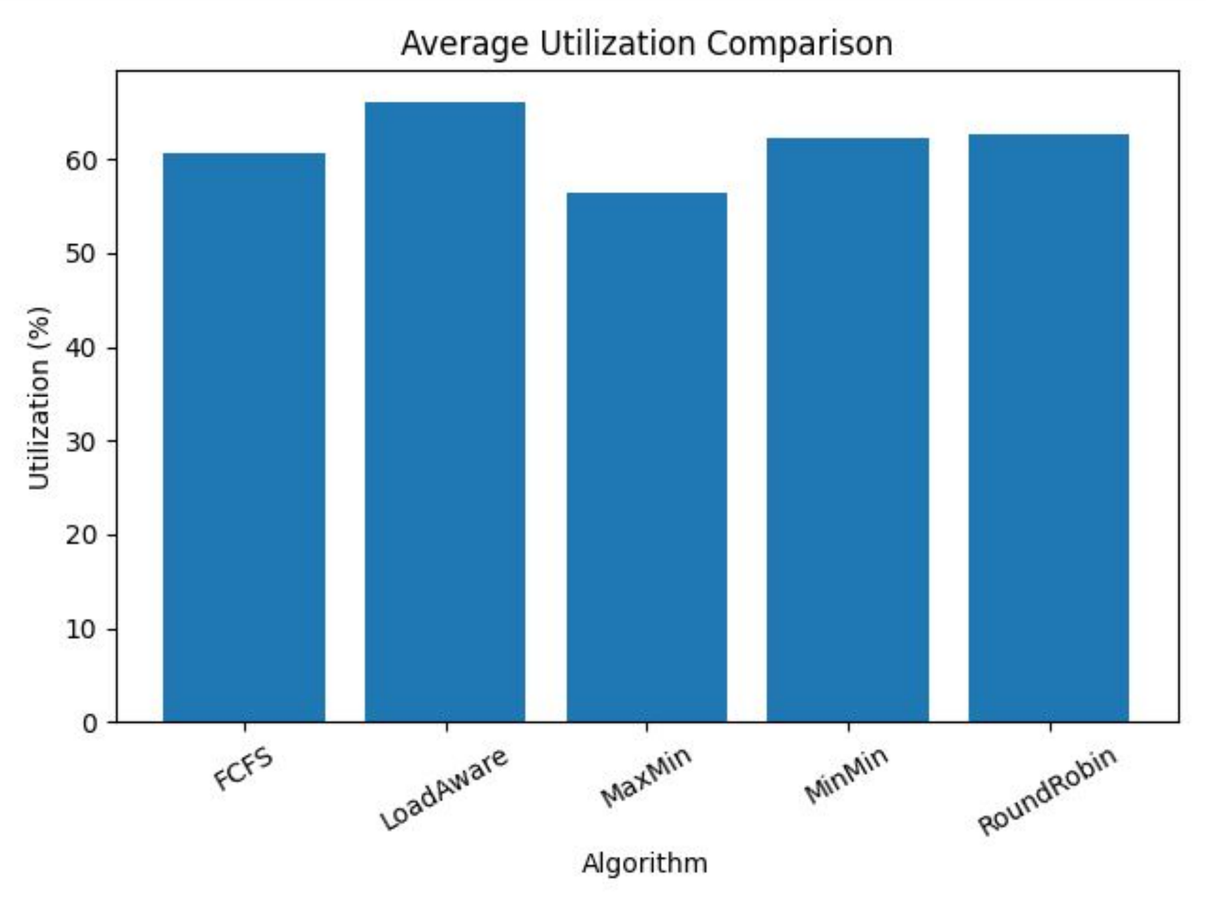
Why this happens?

- **Speedup** depends on how effectively processors are utilized in parallel
- **LoadAware** reduces idle time → more parallel work executed simultaneously
- **MaxMin** creates bottlenecks → limits concurrent execution
- **RoundRobin** ensures fairness but does not fully optimize execution time

LoadAware achieves the highest speedup due to balanced workload distribution. However, gains diminish as processors increase due to dependency constraints, consistent with Amdahl's Law.

Results — CPU Utilization (Efficiency)

Fraction of time processors are doing useful work · higher is better



Observations

- **LoadAware** achieves the **highest utilization** (~66%), indicating better processor usage
- **RoundRobin** and **MinMin** show **moderate utilization** (~62–64%)
- **MaxMin** has the **lowest utilization** (~56%), due to uneven workload distribution
- Overall utilization remains well below 100%, showing limitations in parallel execution

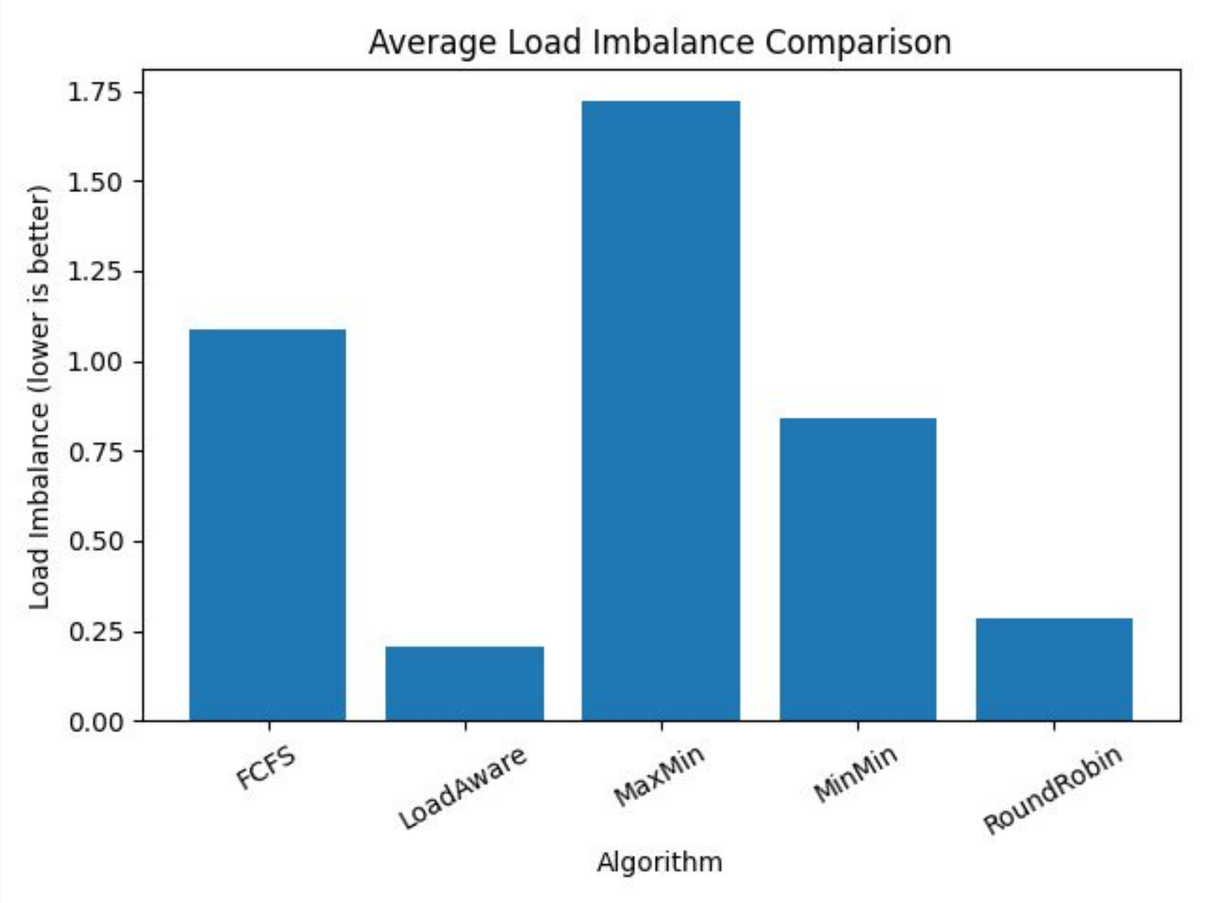
Why this happens?

- Task dependencies prevent all processors from working simultaneously
- Some processors remain idle while waiting for prerequisite tasks
- LoadAware minimizes idle time through balanced scheduling
- MaxMin creates imbalance → some processors overloaded, others idle

Utilization decreases as dependency constraints increase, leading to processor idle time. LoadAware maintains the highest utilization by balancing workload, while MaxMin performs worst due to imbalance.

Results — Load Imbalance

Work distribution across processors · lower = more balanced



Observations

- **LoadAware** achieves the **lowest load imbalance** (~0.23–0.29), indicating near-uniform workload distribution
- **RoundRobin** also performs **well** (~0.28), due to cyclic task assignment
- **MinMin** and **FCFS** show **moderate imbalance**, as they do not consider processor load explicitly
- **MaxMin** has the **highest imbalance** (~1.7+), indicating poor distribution of tasks

Why this happens?

- **LoadAware** assigns tasks based on current processor load, ensuring balanced execution
- **RoundRobin** distributes tasks evenly but ignores execution time differences
- **MaxMin** prioritizes large tasks → some processors overloaded while others remain idle
- **FCFS** and **MinMin** lack load awareness → uneven workload distribution

LoadAware achieves near-perfect load balance, ensuring efficient processor utilization. In contrast, MaxMin creates significant imbalance by prioritizing long tasks, leading to idle processors and reduced performance.

Master Comparison Table

Averaged across all 9 configurations (3 task counts × 3 processor counts)

Algorithm	Avg Makespan ↓	Avg Speedup ↑	Avg Utilization ↑	Load Imbalance ↓	Throughput ↑	Overall Rank
FCFS	1,503.1	2.32x	60.78%	1.09	0.0665 t/u	3rd
RoundRobin	**1,427.9**	**2.45×**	**61.88%**	0.34	**0.0700 t/u**	2nd
MinMin	1,520.9	2.20×	59.64%	1.07	0.0657 t/u	4th
MaxMin	**1,725.9**	**2.08×**	**55.36%**	**2.23**	**0.0579 t/u**	**5th**
LoadAware	**1,420.9**	**2.48×**	61.81%	**0.292**	**0.0704 t/u**	**1st**

** Throughput = avg task count / avg makespan (tasks per time unit)*

LoadAware wins 5 of 5 metrics. RoundRobin is the second best after LoadAware. MaxMin is worst in every single metric.

Key Findings

Data-backed conclusions from experiments

#01

LoadAware = Best Overall

Achieves best performance across all metrics: ~1,420 makespan, ~2.48× speedup, ~61–66% utilization, and lowest imbalance (~0.29). Dynamic load-based scheduling minimizes idle time.

#02

LoadAware = Best Utilization

Highest utilization achieved due to balanced processor workload. Ensures minimal idle time compared to other algorithms.

#03

MaxMin = Worst Performer

High imbalance (~2.2+) and lowest utilization (~55%) due to long-task prioritization causing processor bottlenecks.

#04

Dependencies Limit Parallelism

Task dependencies restrict simultaneous execution, leading to processor idle time and reduced efficiency.

#05

Diminishing Returns with More Processors

Speedup gains reduce as processors increase (Amdahl’s Law), due to sequential components and scheduling overhead.

#06

Synchronization Overhead Exists

Parallel execution requires coordination, which can introduce minor overhead and limit scalability.

Conclusions

What We Proved

- LoadAware achieves the best overall performance, with lowest makespan, highest speedup, highest utilization, and minimal load imbalance
- RoundRobin performs consistently well, but lacks load-awareness optimization
- MaxMin performs worst across all metrics, due to long-task prioritization causing severe imbalance
- Increasing processors improves speedup, but gains diminish due to inherent sequential components (Amdahl's Law)
- Task dependencies limit parallelism, leading to processor idle time and reduced utilization

Limitations

- Synthetic task generation may not reflect real-world workload characteristics
- Centralized scheduling approach introduces coordination overhead and limits scalability
- Homogeneous processor assumption (equal speed and capability) simplifies real-world complexity
- No task preemption, so long-running tasks cannot be interrupted or rescheduled
- Static batch model — does not support dynamic task arrivals or real-time scheduling

Future Work

- Implement work-stealing scheduling for dynamic load balancing across processors
- Extend to heterogeneous systems with speed-aware task allocation
- Evaluate using real-world DAG benchmarks (e.g., PARSEC, SPLASH-2)
- Apply machine learning models to predict optimal scheduling strategy per workload
- Develop distributed scheduling (MPI + OpenMP hybrid) for large-scale systems